

HiDiTingV100 软件

开发指南

文档版本 00B01

发布日期 2025-06-05

前言

概述

本文档系统性描述 HiDiTingV100 芯片解决方案 SDK 的所有发布组件，方便开发者基于 SDK 发布包进行开发调试。

基于本解决方案的整体命名，后续所有文档中，将主 SOC 称为 Brandy，4G CAT1 模块称之为 Hi2131E，GNSS 模块称之为 GS21。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示如不可避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
 须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....	i
1 SDK 发布概述.....	1
1.1 ReleaseDoc 目录概述.....	1
1.2 software 目录概述.....	2
1.3 tools 目录概述.....	3
2 编译构建环境.....	5
2.1 编译构建环境.....	5
2.1.1 操作系统.....	5
2.1.2 依赖软件.....	5
2.1.3 编译工具链.....	6
2.2 编译系统注意事项.....	6
3 SDK 开发指导.....	7
3.1 编译.....	7
3.1.1 启动编译.....	8
3.1.2 特性配置.....	9
3.1.2.1 硬件配置特性.....	9
3.1.2.2 串口方案特性.....	11
3.1.2.3 蓝牙相关特性.....	12
3.1.2.4 媒体相关特性.....	12
3.1.2.5 音频相关特性.....	13
3.1.2.5.1 3A 算法.....	13
3.1.2.5.2 SmartPA 驱动.....	14
3.1.2.5.3 AMR-WB 编解码.....	17

3.1.2.6 低功耗相关特性.....	18
3.1.2.7 驱动相关特性	18
3.1.2.8 32k 时钟源相关特性.....	19
3.1.2.9 Nor Flash 部署	20
3.1.3 各组件编译描述.....	21
3.1.4 编译结果获取	21
3.1.5 Bin 文件描述	21
3.1.6 target 简介.....	22
3.1.7 三方库集成编译.....	22
3.2 系统适配对接	23
3.2.1 系统启动适配	23
3.3 调试环境.....	24
3.3.1 JLink 环境调试	24
3.4 烧录	24
3.4.1 烧录环境说明	24
3.4.2 烧录步骤说明	25
3.5 代码部署.....	25
4 开发调试工具应用	26
4.1 工具概述.....	26
4.2 基础开发工具应用介绍.....	29
4.3 基础维测工具应用介绍	30
4.4 业务应用工具介绍	30
5 RTT 调试使用指导	32
5.1 RTT 源码获取.....	32
5.2 RTT 模块添加.....	33
5.3 RTT Viewer 设置.....	35

插图目录

图 3-1 添加编译选项	10
图 3-2 在 defines 属性内添加宏	11
图 3-3 ADC 自动采样开启示意图	18
图 3-4 添加编译选项切换 RC 32k 时钟源	20
图 3-5 代码示例	25
图 4-1 DevEco Device Tool 工具界面	27
图 4-2 消息插件界面示例	28
图 4-3 烧写完成示意图	29
图 5-1 文件列表	32
图 5-2 APP target 修改	33
图 5-3 SSB target 修改	34
图 5-4 RTT 链接配置示意图	36
图 5-5 RTT 界面打印信息	37

表格目录

表 2-1 编译环境操作系统	5
表 2-2 软件依赖列表	5
表 3-1 编译任务简介	8
表 3-2 串口编译宏及方案	11
表 3-3 蓝牙特性编译宏及方案	12
表 3-4 编译 target 与 bin 文件对应关系	21
表 3-5 二进制获取路径	21
表 3-6 bin 文件说明	21

1 SDK 发布概述

HiDiTing 解决方案 SDK 包含 ReleaseDoc 目录、customer 目录，其中 customer 目录下辖 software 目录以及 tools 目录。

- ReleaseDoc 目录下存放系统软件各子系统开发指南、说明文档以及 API 手册。
HiDiTing 解决方案资料包含 3 个主要芯片模块的资料内容，分别为主 SOC 资料目录 (Brandy)、4G CAT1 资料目录 (Hi2131E) 以及 GNSS 资料目录 (GS21)。分别描述了三款芯片相关的使用和操作指南。其中 4G CAT1 芯片因为涉及独立烧录，在对应目录提供了加载烧录指导。
- software 目录下分为 bin 目录以及 code 目录，其中 bin 目录用于存放发布以及系统启动需要的二进制文件，code 目录用于存放 SDK 发布代码。
- tools 目录用于存放系统编译、开发、调试、资源压缩、维测、产测等环节所用到的工具。

1.1 ReleaseDoc 目录概述

1.2 software 目录概述

1.3 tools 目录概述

1.1 ReleaseDoc 目录概述

ReleaseDoc 目录用于存放系统软件各子系统的开发指南以及 API 手册。

开发指南主要包含如下大类以及内容，通过 pdf 或者 word 格式分别提供：

- 业务领域开发指南：UIKit 开发指南、多媒体软件开发指南、BT/BLE 软件开发指南、lwIP 调测指南、OpenHarmony 组件开发\Slice 开发\JS 应用开发 用户指南。

- 系统功能领域开发指南：Boot 开发指南、FOTA 开发指南、低功耗软件开发指南、安全软件开发指南、文件系统使用指南、NV 存储用户指南。
- 驱动开发指南：设备驱动开发指南、音频驱动开发指南、图形驱动开发指南。
- 工具使用指南：IDE 工具使用指南、DebugKits 工具使用指南、BurnTool 工具使用指南、图形工具使用指南、HiSpark Studio 开发工具使用指南。
- 4G 领域指南：AT 命令 用户指南、VoLTE 开发指南、维测功能 指导书、NV 用户指南。
- GNSS 领域开发指南：GNSS 接口协议 开发指南、GNSS 集成 开发指南、XGNSS 开发指南。

API 手册为各业务、驱动、系统、工具等领域的 API 集合，用于 API 功能查询以及参数查询。全量子系统的标准 API 手册。

1.2 software 目录概述

software 目录主要存放 SDK 软件代码、工程以及系统 A 核、BT 核、DSP 核、GNSS 系统、4G CAT1 系统等分别启动所需要的二进制版本文件。其中除 A 核以外，其他核以及子系统均不支持基于发布的 SDK 代码进行增量开发以及修改。用户可以基于 A 核版本进行增量开发。

software 目录包含 code 目录以及 bin 目录，code 目录用于存放 SDK 发布的软件代码，bin 目录用于存放二进制 bin 文件。

bin 目录包含如下二进制 bin 文件：

ssb.bin	boot 启动加载所需二进制文件。
application.bin	A 核系统版本二进制文件，可通过 code 目录 SDK 版本增量开发后生成。
bt.bin	BT 核系统二进制文件，固定版本发布，不支持增量开发。
dsp_main.bin, dsp_overlay.bin	DSP 核系统二进制文件，固定版本发布，不支持增量开发。
gnss_cfg.bin	GNSS 系统二进制文件，固定版本发布，不支持增量开发。
caxx_cfg.bin	NFC 系统二进制文件，非版本必选项，固定版本发布，不

	支持增量开发。
Hi2131EV100.fwpkg	4G CAT1 加载包，固定版本发布，不支持增量开发。

code/sdk 目录包含如下子目录：

application	应用目录，支持用户在此目录下增量开发上层系统内容以及应用内容。全系统启动的代码也在此目录下。当前 SDK 会在此目录下提供部分 sample 代码用于客户开发参考。
bootloader	用于存放系统启动过程中二进制文件加载流程的代码。
build	编译构建工程代码。
drivers	设备驱动代码，提供 BSP 驱动代码。
include	公共头文件。
interim_binary	软件库、二进制文件存放目录。
kernel	操作系统以及操作系统功能封装代码。
middleware	中间件代码，提供系统框架代码。
open_source	开源相关组件代码。
output	编译构建版本生成目录（发布的 SDK 下无此目录，用户构建后生成）。
tools	编译工具链。
vendor	闭源数学库。
build.py	编译入口脚本。
CMakeLists.txt	cmake 编译公共文件。

1.3 tools 目录概述

tools 目录用于存放开发、调试以及版本烧录的多个工具链，当前提供如下四种工具内容：

DebugKits	提供日志打印解析、定制命令输入、内存导出解析等功能。
BurnTool	提供版本烧录功能。
DevEco	开发工具，集成一键式编译、烧录、单步调试以及内存查询，变量监控等功能。
HiSpark Studio 工具	开发工具，集成一键式编译、烧录、单步调试以及内存查询、变量监控，并提供 UIKit 模拟器功能。

更多详细工具使用、安装功能请参考对应工具使用指南文档。

2 编译构建环境

HiDiTing SDK 支持开发者模式编译系统：开发者模式编译系统主要通过 HiDiTing SDK 提供全套的编译框架和编译系统，用户通过 IDE 添加文件或通过适配修改自身编译系统的方式，来适配 Brandy SDK 的编译系统。

2.1 编译构建环境

2.2 编译系统注意事项

2.1 编译构建环境

2.1.1 操作系统

支持 Linux 环境下以及 Windows 环境下进行编译环境部署以及版本编译。对应操作系统版本支持见表 2-1。

表2-1 编译环境操作系统

操作系统	操作系统版本
Linux	Ubuntu 20.04
Windows	Windows 10

2.1.2 依赖软件

表2-2 软件依赖列表

软件名称	版本信息
------	------

软件名称	版本信息
python	3.8.0 以上。同步需要通过 pip 安装 kconfiglib、pycparser 依赖包。
Cmake	3.14.1 以上。
ninja	SDK 发布，仅 Windows 下编译需要对应 ninja.exe 文件，需要配置环境变量。

2.1.3 编译工具链

系统提供基于 Windows 以及 Linux 两套编译工具链，以提供 llvm 编译能力：

linx-llvm-binary-release-musl	Linux 编译工具链。
linx-llvm-binary-release-win-musl	Window 编译工具链。

路径：software\code\sdk\tools\bin\compiler\linx\linx_170\linx-llvm-binary-debug

2.2 编译系统注意事项

HiDiTing 解决方案 SDK 提供基于 IDE 工具的 Windows 下编译能力，提供基于 Windows/Linux 系统下的 python 入口编译能力。全系统编译均使用 python+cmake+llvm 编译器的编译能力，支持 Windows/Linux 跨操作系统编译。需要注意的是，Windows 下编译需要 ninja 库支持，对应 ninja.exe 也会在 SDK 包中同步提供。

3 SDK 开发指导

- 3.1 编译
- 3.2 系统适配对接
- 3.3 调试环境
- 3.4 烧录
- 3.5 代码部署

3.1 编译

CMake 编译系统框架描述:

```
-- cmake
|   |-- build_component.cmake           // 组件编译相关
|   |-- build_function.cmake           // 编译系统公共函数
|   |-- build_linker.cmake             // 链接脚本处理
|   |-- global_variable.cmake          // 初始化全局变量, 避免因为增量编译导致
CmakeCache 膨胀
-- config                               // 构建及工具配置
|   |-- target_config // 编译target 配置
|       |-- common_config.py           // 构建公共参数配置, 存放公共宏、公共编译选项、
组件集合以及和 arch 相关的各种配置
|       |-- target_config.py           // target 模板配置
```

```

|      |-- config.py          // 项目具体的配置， 最终的配置为模板配置 + 具体配置
|
|      |-- build_utils.py     // 构建脚本公共函数
|
|      |-- cmake_builder.py   // 组件构建命令
|
|      |-- enviroment.py      // 用于接收和解析命令， 根据target_name 组织各种编译参数
|
|-- toolchains                // 存放工具链
|
|      |-- gcc-arm-none-eabi-10-2020-q4.cmake // 编译工具链配置文件
|
|      |-- hcc_riscv32_1.2.0.b045.cmake
|-- README.md

```

3.1.1 启动编译

编译 target 介绍

基于 HiDiTing 整体解决方案的对标场景以及组件组合，当前提供了 3 个编译 target 供用户使用。此 3 个 target 定义于 “software\code\sdk\build\config\target_config\brandy” 目录下 “config.py” 文件中。

表3-1 编译任务简介

编译任务名称	编译任务介绍	配套组件组成
brandy-ssb-diting	用于 boot 流程启动的 ssb.bin 镜像构建。	-
brandy-diting	用于完整 HiDiTing 解决方案版本包构建。	liteos + Ulkit + NandFlash + MIPI + psram + Native-JS 应用框架+GNSS+CAT1
brandy-diting-nfc	用于完整 HiDiTing 解决方案版本包构建 + NFC 套片（可选）解决方案版本包构建。	liteos + Ulkit + NandFlash + MIPI + psram + Native-JS 应用框架+GNSS+CAT1+NFC

cmake 编译

基于 IDE 的编译，请参考《HiDiTingV100 IDE 工具 使用指南》，采取工程式一键编译，本文档中不做赘述。当前描述基于 CMake 的解决方案 SDK 编译方式。

Linux 下 cmake 编译，在安装好对应编译依赖软件后，在 Linux 下运行 “build.py” 脚本进行编译。整体步骤如下：

步骤 1 在 SDK 根目录下，通过执行 “./build.py brandy” 指令，运行 “build.py” 脚本。

步骤 2 选择对应编译 target，输入对应 target 编号即可。

步骤 3 运行结束后在 output 目录下获取编译后生成的二进制文件，整体编译完成。

----结束

Windows 下 cmake 编译，在安装好对应编译依赖软件后，在 “tools\DevEco\DevTools_Brandy_v1.3” 下双击运行 “env_start.bat” 脚本，整体步骤如下：

步骤 1 进入 “tools\DevEco\DevTools_Brandy_v1.3” 目录，双击运行 “env_start.bat” 脚本。

步骤 2 在命令行指令框内，通过输入 Windows 的 “CD: ” 命令，将命令行盘符切换至 “software\code\sdk” 目录，执行 “python build.py -ninja -c” 指令，同样会产生对应编译 target 列表，输入对应 target 列表编号即可开始编译，编译列表同 Linux 下编译列表。

步骤 3 运行结束后在 output 目录下获取编译后生成的二进制文件，整体编译完成。

----结束

3.1.2 特性配置

3.1.2.1 硬件配置特性

当前编译系统提供了对应的编译选项以方便用户在不同硬件设备间进行驱动切换。

当前硬件设备切换共分为两类：

- 用于提供 EMMC 与 NandFlash 以及对应文件系统的切换，通过 ‘CFG_DRIVERS_NANDFLASH’ 和 ‘CFG_DRIVERS_MMC’ 两个编译选项进行控制。此两个编译选项为互斥关系。

- 用于提供 QSPI 显示通路以及 MIPI 显示通路以及配套表盘 TP 驱动的切换，通过 'QSPI_DISPLAY' 和 'TPTYPE_TMA525B' 两个编译选项联合控制。
 - 当 'QSPI_DISPLAY'、'TPTYPE_TMA525B' 均设置时，则提供 QSPI 显示通路。
 - 当 'QSPI_DISPLAY'、'TPTYPE_TMA525B' 均不设置时，则默认提供 MIPI 显示通路。
 - 圆形屏和方形屏设置：当打开宏 'SUPPORT_GPU_GMMU' 时为圆形屏，当关闭宏 'SUPPORT_GPU_GMMU' 时为方形屏。
 - 选择使用 QSPI 显示通路时，需要关闭 MIPI 相关宏 'MIPI_ULPS_SUPPORT'。

具体设置编译选项方法：

1. 进入 "software\code\sdk\build\config\target_config\brandy" 目录，打开 "config.py" 。
2. 如需切 EMMC 与 NandFlash，按照图 3-1 所示添加对应编译选项。

图3-1 添加编译选项



```

'brandy-liteos': {
    'board': 'evb',
    'base_target_name': 'target_standard_brandy_application_template',
    'defines': ['PRE_ASIC', 'BRANDY_PRODUCT_EV8', 'VERSION_STANDARD', 'LITEOS', 'CONFIG_OTA_UPDATE_SUPPORT',
    'ALL_SOURCE', 'SUPPORT_CXX', 'I2C_SLAVE_REG_ADDR_4BYTE', 'CONFIG_PSRAM_SUPPORT',
    'TARGET_CHIP_BRANDY=1', 'BRANDY_CHIP_V100-1', 'CFG_DRIVERS_MMC', 'CONFIG_ZDIAG_NV_SUPPORT',
    'CONFIG_ZDIAG_AUDIO_PROC_SUPPORT', 'CONFIG_ZDIAG_AUDIO_GWUP_SUPPORT', 'CONFIG_ZDIAG_AUDIO_PROBE_SUPPORT',
    'SUPPORT_BLE', 'SUPPORT_BREDR', 'ENABLE_UKIT', 'SUPPORT_GPU_JPEG',
    'SUPPORT_GPU_GMMU', 'SUPPORT_GPU_OPENVG', 'SW_UART_DEBUG', 'MIPI_ULPS_SUPPORT'],
    'ram_component': ['algorithm', 'dfx_gport_brandy', 'dfx_update', 'dfx_nv', 'osat', 'arch_port', '-:testsuite',
    'liteos_port', 'lcd', 'qspi_display',
    'psram', 'hal_mipi', 'mipi_tx', 'non_os', 'touch',
    'ulp_aon', 'hal_l2ram',
    'x_dpai', 'x_vfs', 'x_disk', 'x_fat', 'drv_mmc', 'fs_yaffs2',
    'pm_service', 'pm_brandy', 'cam_header', 'bg_cmd', 'gnaphic_at_service', 'app_at_service',
    'audio_at_service', 'media_at_service', 'bt_mnagenet_service',
    'audio_proc', 'audio_dump', 'audio_probe'],
    'ram_component_set': ['bgh', 'media', 'gpu', 'graphic_uikit', 'graphic_test', 'bgh_audio', 'dfx_set'],
    'dsp_version': 'normal',
    'packet': True
},
'brandy-liteos-yaffs': {
    'board': 'evb',
    'base_target_name': 'target_standard_brandy_application_template',
    'defines': ['PRE_ASIC', 'BRANDY_PRODUCT_EV8', 'VERSION_STANDARD', 'LITEOS', 'CONFIG_OTA_UPDATE_SUPPORT',
    'CONFIG_ZDIAG_NV_SUPPORT', 'ALL_SOURCE', 'SUPPORT_CXX', 'I2C_SLAVE_REG_ADDR_4BYTE', 'CONFIG_PSRAM_SUPPORT',
    'TARGET_CHIP_BRANDY=1', 'BRANDY_CHIP_V100-1', 'CFG_DRIVERS_NANDFLASH', 'SUPPORT_BLE', 'SUPPORT_BREDR',
    'ENABLE_UKIT', 'SUPPORT_GPU_JPEG', 'SUPPORT_GPU_GMMU', 'SUPPORT_GPU_OPENVG',
    'SW_UART_DEBUG', 'MIPI_ULPS_SUPPORT'],

```

说明

'CFG_DRIVERS_NANDFLASH' 与 'CFG_DRIVERS_MMC' 互斥，仅能支持一个。

图例解释：

- target 'brandy-liteos'支持 EMMC。
- target 'brandy-liteos-yaffs'支持 NandFlash。

3. 如下 target 默认是 MIPI 显示通路，如需切换 QSPI 显示通路，请在如图 3-2 所示的 defines 属性内添加 'QSPI_DISPLAY' 与 'TPTYPE_TMA525B'，且删除 MIPI 属性编译宏 'MIPI_ULPS_SUPPORT'。

图3-2 在 defines 属性内添加宏

```
'brandy-liteos': {
  'board': 'evb',
  'base_target_name': 'target_standard_brandy_application_template',
  'defines': [ 'PRE_ASIC', 'BRANDY_PRODUCT_EVB', 'VERSION_STANDARD', '__LITEOS__', 'CONFIG_OTA_UPDATE_SUPPORT',
    'ALL_SOURCE', 'SUPPORT_CXX', 'I2C_SLAVE_REG_ADDR_4BYTE', 'CONFIG_PSRAM_SUPPORT',
    'TARGET_CHIP_BRANDY=1', 'BRANDY_CHIP_V100=1', 'CFG_DRIVERS_MMC', 'CONFIG_ZDIAG_NV_SUPPORT',
    'CONFIG_ZDIAG_AUDIO_PROB_SUPPORT', 'CONFIG_ZDIAG_AUDIO_DUMP_SUPPORT', 'CONFIG_ZDIAG_AUDIO_PROBE_SUPPORT',
    'SUPPORT_BLE', 'SUPPORT_BREDR', 'ENABLE_UKIT', 'SUPPORT_GPU_JPEG',
    'SUPPORT_GPU_GMMU', 'SUPPORT_GPU_OPENVG', 'SW_UART_DEBUG', 'MIPI_ULPS_SUPPORT',
  ],
  'ram_component': [ 'algorithm', 'dfx_port_brandy', 'dfx_update', 'dfx_nv', 'osal', 'arch_port', '-:testsuite',
    'liteos_port', 'lcd', 'qspi_display',
    'psram', 'hal_mipi', 'mipi_tx', 'non_os', 'touch',
    'ulp_aon', 'hal_l2ram',
    'x_dpai', 'x_vfs', 'x_disk', 'x_fat', 'drv_mmc', 'fs_yaffs2',
    'pm_service', 'pm_brandy', 'cmn_header', 'at_cmd', 'graphic_at_service', 'app_at_service',
    'audio_at_service', 'media_at_service', 'bt_manager_at_service',
    'audio_proc', 'audio_dump', 'audio_probe',
  ],
  'ram_component_set': [ 'bgh', 'media', 'gpu', 'graphic_uikit', 'graphic_test', 'bgh_audio', 'dfx_set'],
  'dsp_version': 'normal',
  'packet': True
},
```

说明

'QSPI_DISPLAY' 与 'TPTYPE_TMA525B' 两个编译选项务必要同时存在或删除。

3.1.2.2 串口方案特性

表3-2 串口编译宏及方案

编译宏	产测方案	适配场景
'FT_SINGLE_UART'	单 UART 方案	适用于 MCU 仅剩余一个 UART 口供 AT 命令通信使用。
'FT_BT_UART'	BT UART 方案	适用于 MCU 无 UART 口供 AT 命令通信使用。

- 单 UART 方案需要打开编译宏 'FT_SINGLE_UART'，打开后原 AT 口与 HSO 口二合一为 HS_UART 口，波特率为 500000，此 UART 口同时支持 DebugKits 工具和 AT 命令，但要求芯片启动完成后，再连接 Debugkits 工具。
- BT UART 方案需要打开编译宏 'FT_BT_UART'，打开后原 AT 口与 HSO 口不再使用（注：当前版本未去掉原有 UART 口的初始化配置）。上电后使用烧录口进行烧录，完成后烧录 UART 口复用给外设。芯片启动后 AT 命令默认使用 BT 核的 UART2 (S_AGPIO_L9、S_AGPIO_L10) 口，波特率 115200，低功耗模式下需

要先发送（最少 30Byte）任意字符串对 UART 口进行唤醒，然后使用原有 AT 命令即可实现命令下发。

说明

串口方案特性亦用于产线测试，具体测试项及操作步骤可以参照《HiDiTingV100 产线 测试指南》。

3.1.2.3 蓝牙相关特性

蓝牙当前版本支持通过编译宏和增删组件的方式来配置单模 BLE 和双模 BLE/BT 特性，具体在 “software\code\sdk\build\config\target_config\brandy\config.py” 中的 target 配置。

表3-3 蓝牙特性编译宏及方案

蓝牙特性方案	宏 'SUPPORT_BLE'	宏 'SUPPORT_BREDR'	组件 'bgh_audio'
单 BLE 特性	打开	关闭	删除
双模 BLE/BT 特性	打开	打开	增加

说明

蓝牙当前版本只支持以上两种特性方案。

3.1.2.4 媒体相关特性

媒体组件支持裁剪，相关特性集合有 'media'、'media_mini'。

'media' 支持音视频相关特性，'media_mini' 只支持音频相关特性，具体请参见 “software\code\sdk\build\config\target_config\common_config.py”。

可以在 “software\code\sdk\build\config\target_config\brandy\config.py” 文件中 target 的 “ram_component_set” 中配置媒体相关的特性集合。

例如：

brandy-target1 里面配置的是 'media_mini'。

brandy-target3/brandy-target5 里面配置的是 'media'。

须知

因为 'media_mini' 是 'media' 的特性子集，所以同一个 target 不能同时配置 'media' 和 'media_mini'。

3.1.2.5 音频相关特性

3.1.2.5.1 3A 算法

3A 算法参数配置

3A 算法参数记录在 "middleware\chips\brandy\nv\nv_config\cfg\acore\app.json" 文件中，分为以下两套参数：

- audio_vqe_param: key_id 为 0x320C，用于通话场景和录音场景。

```
"audio_vqe_param":{
  "key_id": "0x320C",
  "key_status": "alive",
  "structure_type": "audio_vqe_param_t",
  "attributions": 1,
  "value": [48, [0, 0, 0, 0, 16843009, 33554689, 16000, 16000,
    1024, 100, 1985229328, 1985229328, 16843009, 257, 257, 256,
    16778116, 1280, 4, 0, 458824, 11796580, 327946, 196608005,
    65576964, 1000, 21499085, 67108874, 327680, 655367, 2126009683,
    654734439, 1, 0, 0, 0, 0, 0, 10551292, 655336, 4286447619, 64536,
    500, 16384, 3277, 0, 66049, 0]]
},
```

- audio_vqe_kws_param: key_id 为 0x320D，用于语言唤醒场景和命令词识别场景。

```
"audio_vqe_kws_param":{
  "key_id": "0x320D",
  "key_status": "alive",
  "structure_type": "audio_vqe_param_t",
  "attributions": 1,
  "value": [48, [0, 0, 0, 0, 16843009, 33554689, 16000, 16000,
    1024, 100, 1985229328, 1985229328, 16843009, 257, 257, 256,
    16778116, 1280, 4, 0, 458824, 11796580, 327690, 13107205,
    65576964, 1000, 679102459, 67108874, 327680, 655367, 2126009683,
    654734439, 1, 0, 0, 0, 0, 0, 10551272, 655336, 4286447619, 64536,
    500, 16384, 3277, 0, 66049, 0]]
},
```

value 成员里的数组即为 3A 算法参数。需要更新时，只需将新参数替换掉旧参数，重新编译即可。

3A 算法 bypass

产线测试时，为了得到麦克风录制的原始数据，此时需要将 3A 算法 bypass，需要在 "middleware\chips\brandy\nv\nv_config\cfg\acore\app.json" 文件中，增加以下代码：

```
"product_test_mode":{
```

```
"key_id": "0x4109",  
"key_status": "alive",  
"structure_type": "uint8_t",  
"attributions": 1,  
"value": 1  
}
```

value 成员里的 1 表示打开产测模式，此时 3A 算法会被 bypass；如果需要使用 3A 算法，请将 value 置为 0。

3.1.2.5.2 SmartPA 驱动

SmartPA 适配

SDK 提供外置 Codec 适配示例，版本提供的示例为 SmartPA 的适配。

SmartPA 芯片驱动的适配

- 版本默认初始化的是 EVB 板子的 AW88262，同时也提供了 AW88166 适配示例，可以通过宏开关切换。
- SmartPA 驱动初始化入口：application\brandy\brandy_standard\thread_pri_init.c。
- SmartPA 宏开关选择型号的代码片段如下：需要根据实际情况打开相应的 SmartPA 特性宏，关闭其他无关型号特性宏，同时根据对接的 I2S 端口进行正确配置。

```
/* Audio codec vendor select according to hardware connection:  
 * define AUDIO_SMARTPA_AW88166 : macro to init smartpa aw88166;  
 * define AUDIO_SMARTPA_AW88166_I2S_ID UAPI_SND_OUT_PORT_I2S0 : macro to  
specify i2s id connecting to smartpa aw88166;  
 */  
#define AUDIO_SMARTPA_AW88262  
#define AUDIO_SMARTPA_AW88262_I2S_ID UAPI_SND_OUT_PORT_I2S2
```

- SmartPA 驱动的基本流程：
 - 驱动对外一共五个接口：具体参考
“application\audio\vendor\aw88166\include\aw88166.h”。
 - 驱动的 init/deinit 接口跟随 MCU 的上电和下电流程调用。
 - 驱动的 set_aef_profile 接口跟随业务的输出音效配置接口回调
(uapi_snd_set_aef_profile)。

- 驱动的 start/stop 接口跟随下行通路的创建和销毁调用，通过 uapi_audio_vendor_register 接口注册个音频下行模块；stop 接口会让 SmartPA 进入低功耗待机状态。

示例参考代码如下：

application\brandy\brandy_standard\thread_pri_init.c:145

```
static uapi_vendor_driver g_aw88166_driver = {
    .out_port = AUDIO_SMARTPA_AW88166_I2S_ID,
    .init = audio_aw88166_start,
    .deinit = audio_aw88166_stop,
    .set_aef_profile = audio_aw88166_set_aef_profile,
};

static td_s32 audio_aw88166_register_driver(void)
{
    const uapi_vendor_codec_attr attr = {};

    audio_aw88166_init(&attr); /* Init aw88166 once when device power on */

    return uapi_audio_vendor_register(&g_aw88166_driver);
}
```

- 将新增的驱动文件增加到“application\audio\vendor\CMakeLists.txt”中，如图所示（需要删去图中默认的 aw88262 的驱动文件，替换为新增的驱动文件）。

```
32 if("${CFG_SAP_VENDOR_DRIVER_AW88262_SUPPORT}" STREQUAL "y")
33 set(COMPONENT_SRC ${COMPONENT_SRC}
34     ${CMAKE_CURRENT_SOURCE_DIR}/aw88262/aw88262.c
35 )
36 set(COMPONENT_INC ${COMPONENT_INC}
37     ${CMAKE_CURRENT_SOURCE_DIR}/aw88262
38 )
39 endif()
```

该部分需要删去

注意需要把所有新增的源文件及头文件路径都加上，以 AW88166 为例：

```
set(COMPONENT_SRC ${COMPONENT_SRC}
    ${CMAKE_CURRENT_SOURCE_DIR}/aw88166/aw88166.c
    ${CMAKE_CURRENT_SOURCE_DIR}/aw88166/aw883xx.c
    ${CMAKE_CURRENT_SOURCE_DIR}/aw88166/aw883xx_calib.c
    ${CMAKE_CURRENT_SOURCE_DIR}/aw88166/aw883xx_device.c

    ${CMAKE_CURRENT_SOURCE_DIR}/aw88166/product_init_files/aw88166/aw883xx_pid_20
66_init.c
)
set(COMPONENT_INC ${COMPONENT_INC}
    ${CMAKE_CURRENT_SOURCE_DIR}/aw88166
    ${CMAKE_CURRENT_SOURCE_DIR}/aw88166/product_init_files/aw88166
    ${CMAKE_CURRENT_SOURCE_DIR}/aw88166/config/aw88166/mono/16bit
```


)

- 适配 SmartPA 驱动的注意事项：以 AW88166 为例。
 - 根据硬件连接情况，正确配置 SmartPA 芯片的上电管脚。
 - 根据硬件连接情况，正确配置 SmartPA 芯片的 I2C 通信的序号。
 - 根据硬件连接情况以及芯片手册，正确配置 SmartPA 芯片的 I2C 设备地址。

```
application\audio\vendor\aw88166\aw88166.c:21
#define AW88166_PWR_ON_PIN    S_MGPIO4
#define AW88166_I2C_BUS_ID    I2C_BUS_0
#define AW88166_I2C_DEV_ADDR 0x34
```

SmartPA 音效算法的适配

SmartPA 音效算法的适配需要提供整机到 SmartPA 芯片厂商调音，刷新音效算法和音效参数。

SmartPA 有两种：

无 DSP 型：音效处理由 SOC 芯片中的 DSP 完成，例如艾为的 AW88262。

带 DSP 型：音效算法直接在 PA 内部的 DSP 上运行，例如艾为的 AW88166。

- 对于 PA 芯片内部不带 DSP 的型号（如 AW88262），发布版本默认集成了一个艾为提供的音效算法（dsp_overlay.bin 中的 aef.bin）。
 - 算法版本：MiniSKTune_MTK_DSP_HIFI3_AAPI_Porting，版本号为 V0.0.0.19；（需要与艾为确认 PA 选型与该算法版本是否适配）。
 - 算法版本升级或者更换需要参见《HiDiTingV100 AEF overlay 开发指南》重新刷新“aef.bin”。
 - PA 音效调测：为了获取准确的音效算法参数，需要整机厂商提供最终整机给 SmartPA 芯片厂商，用于进行音效参数调节。调节完成后进行参数 bin 的刷新。
 - i. 音效参数 bin 打包在 DSP 镜像包的“dsp_overlay.bin”中。
 - ii. 音效参数 bin 根据业务场景分为 3 个：aef_music.bin、aef_voip.bin、aef_ringtone.bin。要求把 PA 厂商调测好的参数 bin 重命名成这仨参数 bin 替换到“dsp_overlay.bin”中。

举例：MCU 在创建本地音乐通路的时候会在打印 log。

```
"[A][ALERT][ao_get_aef_param_bin_by_profile:701]
<aef_music.bin>"
```

iii. 替换 “dsp_overlay.bin” 中音效参数 bin 的方法:

1) 解包/打包工具:

application/audio/sample_dsp_overlay/package_tool_dsp_overlay.py

直接执行该脚本，会输出脚本的使用指南/help 信息:

```
python3 application/audio/sample_dsp_overlay/package_tool_dsp_overlay.py
```

2) 音效参数替换简略步骤

① 进入路径 “application/audio/sample_dsp_overlay/”。

② dsp_overlay.bin 的解包。

```
./package_tool_dsp_overlay.py unpack max
```

③ 解包在当前路径的 “out/overlay_bin” 目录下。

④ 替换 “out/overlay_bin” 下面的音效参数 bin。

⑤ 用脚本重新打包。

```
./package_tool_dsp_overlay.py packet max
```

⑥ 按往常一样编译镜像包烧录 “dsp_overlay.bin”。

- 对于 PA 芯片内部自带 DSP 的型号 (如 AW88166)，音效算法跑在 SmartPA 芯片内部的 DSP 上，SOC 的 DSP 上的音效算法库需要替换为 bypass 版本，具体请参见《HiDiTingV100 AEF overlay 开发指南》，使用 bypass 版本的 “aef.bin” 替换 “dsp_overlay.bin” 中的 “aef.bin”。
- PA 的音效参数由 PA 的 MCU 驱动下发：PA 驱动的适配层 set_aef_profile 接口就是注册给音频模块根据场景切换音效参数使用。

可以参照 audio_aw88166_set_aef_profile 的具体实现，音频模块会根据业务场景配置的 profile 选择 PA 的音效场景。

所以音效参数就在 PA 供应商的驱动包中：

application/audio/vendor/aw88166/config/aw88166/mono/16bit/

3.1.2.5.3 AMR-WB 编解码

须知

使用 AMR-WB 编解码功能之前，请确保已切换到 16M Flash 版本。

若需要使用 AMR-WB 编解码功能，请在 “build/config/target_config/brandy/config.py” 中将当前使用的 target 的 dsp_version 改为 ultra。

3.1.2.6 低功耗相关特性

具体低功耗相关特性配置请参见《HiDiTingV100 低功耗软件 开发指南》。

3.1.2.7 驱动相关特性

- ADC 自动采样需要开启宏 'CONFIG_ADC_SUPPORT_AUTO_SCAN' 和 'CONFIG_ADC_SUPPORT_LONG_SAMPLE'，手动采样需要关闭这两个宏。

如图 3-3 所示为开启了 ADC 自动采样。

图3-3 ADC 自动采样开启示意图

```
'target_standard_brandy_application_template': {  
  'chip': 'brandy',  
  'core': 'acore',  
  'board': 'evb',  
  'tool_chain': 'clang-riscv-linux',  
  'build_type': 'COMPILE',  
  'hso_enable': True,  
  'os': 'liteos',  
  'CONFIG_WATCHDOG_USING_V100': 'y',  
  'CONFIG_PWM_USING_V150': 'y',  
  'std_libs': ['c++', 'clang_rt.builtins-riscv32', 'gloss', 'c++abi', 'unwind', 'm'],  
  'defines': ['BUILD_APPLICATION_STANDARD', 'HSD_SUPPORT', 'CMD_ENABLE', 'CONFIG_APPS_CORE',  
    'DIAG_PRINT', 'I2C_SLAVE_REG_ADDR_4BYTE', 'LIBAPP_VERSION',  
    'LIBBUILD_VERSION', 'LIBCMD', 'LIBCONNECTIVITY', 'LIBCPU_LOAD', 'LIBCPU_UTILS', 'LIBCRC', 'LIBDFX',  
    'LIBLIB_UTILS', 'LIBLOG', 'LIBLOG_READER', 'LIBOSAL', 'LIBPANIC', 'LIBTEST_COMMON',  
    'LITEOS_ONETRACK', 'MASTER_LOAD_SLAVE', 'SRCPACKGE_BUILD', 'SUPPORT_CXX', 'CONFIG_XIP_ENABLE_READ_OVERTIME',  
    'USE_CMSIS_OS', 'USE_LITEOS', 'VERSION_STANDARD', 'ALL_SOURCE', 'LITEOS',  
    'CONFIG_NV_SUPPORT_SINGLE_CORE_SYSTEM', 'CONFIG_NV_FEATURE_SUPPORT', 'CONFIG_SPI_SUPPORT_I2C',  
    'RISC_V70', 'SUPPORT_DIAG_V2_PROTOCOL', 'CONFIG_PINCTRL_SUPPORT_IE',  
    'DUMP_MEM_SUPPORT', 'DUMP_REG_SUPPORT', 'SUPPORT_DFX_EXCEPTION', 'SUPPORT_DOWNT_RESET_AFTER_EXCEPTION',  
    'CONFIG_ADC_SUPPORT_AUTO_SCAN', 'CONFIG_ADC_SUPPORT_LONG_SAMPLE'],  
  'defines_set': ['chip_defines', 'version_defines'],  
  'ram_component': ['brandy_std_libs', 'liteos_207_1_0', 'partition', 'chip_brandy', 'driver_header', 'nand_flash',  
    'brandy_app_standard', 'board_brandy', 'brandy_clocks', 'pwm_brandy', 'hal_sectors',  
    'partition_brandy', 'nv', 'nv_brandy', 'mtd_cam', 'segger', 'mbedtls'],  
  'ram_component_set': ['std_common_lib', 'mem', 'cpu', 'lpc', 'audio',  
    'xip_os', 'adc', 'ipc', 'spi', 'dma', 'i2c', 'pwm', 'watchdog', 'uart', 'otp',  
    'gpio_v100', 'qspi', 'cpu_trace', 'time_set', 'security', 'efuse_v100',  
    'mem_monitor', 'calendar', 'update_app', 'pinctrl', 'pmp_set', 'button', 'flash'],  
}
```

说明

ADC 自动采样与手动采样互斥。

- SPI 支持轮询模式与 DMA 模式自动切换功能，需要开启宏 'CONFIG_SPI_SUPPORT_DMA' 和 'CONFIG_SPI_SUPPORT_POLL_AND_DMA_AUTO_SWITCH'，并且设置宏 'CONFIG_SPI_AUTO_SWITCH_DMA_THRESHOLD=8'，其中数字 8 代表自动切换轮询与 DMA 模式的切换阈值，当传输大于等于 8Byte 时驱动自动切换为 DMA 传输，小于 8Byte 则使用轮询模式传输，该阈值可按照使用场景进行设置。
- I2C 支持轮询模式与 DMA 模式自动切换功能，需要开启宏 'CONFIG_I2C_SUPPORT_DMA' 和 'CONFIG_I2C_SUPPORT_POLL_AND_DMA_AUTO_SWITCH'，并且设置宏 'CONFIG_I2C_POLL_AND_DMA_AUTO_SWITCH_THRESHOLD=8'，其中数字 8 代表自动切换轮询与 DMA 模式的切换阈值，当传输大于等于 8Byte 时驱动

自动切换为 DMA 传输，小于 8Byte 则使用轮询模式传输，该阈值可按照使用场景进行设置。

须知

当 'QSPI_DISPLAY'、'TPTYPE_TMA525B' 均设置时，即显示为 QSPI 显示通路时，由于 QSPI 屏幕侧器件限制，此时建议关闭 DMA 模式，使用轮询模式，避免 QSPI 显示出现异常。

- NandFlash 器件涉及的 ECC 特性，需要使用 'ENABLE_ECC' 宏在编译时进行配置开启或者关闭。

须知

- ECC 特性首次开启或者关闭时，需要对文件系统格式化后再使用。
- ECC 特性，需要对应版本的 APP、Recovery 等 MCU 版本同步开启或者同步关闭，否则可能导致启动或者升级失败。
- 制作文件系统镜像时，如果 APP 开启或者关闭 ECC 特性，对应 SSB 镜像也需要和其保持同步开启或者关闭。

3.1.2.8 32k 时钟源相关特性

当前编译系统提供了对应的编译选项以方便用户在不同的 32k 时钟源之间进行切换。

当前 32k 时钟源共分为三类：

- RC 32k：版本默认配置。对应编译宏为 'SUPPORT_RC_CALIBRATION'。
- XO 32k：使用前需要进行时钟精度测量，评估满足时钟要求后可使用。
- ULP 32k：仅在关机状态下使用。为提高关机计时准确性，关机之前应调用 `uapi_calendar_calibration_enable()` 方法启用 ULP 32k 的关机自校准功能。

须知

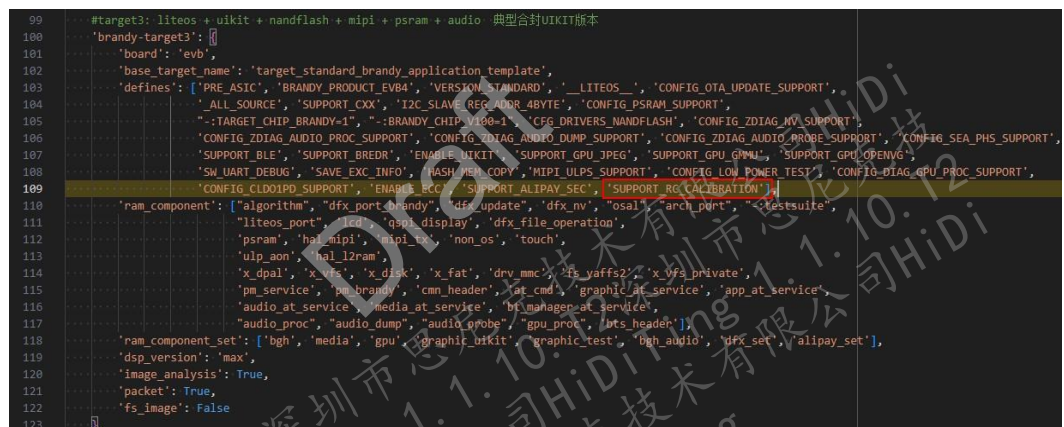
- 由于内置 RC 32k 会在蓝牙流程中进行时钟校准，为确保计时准确，需开启蓝牙广播或保持蓝牙连接。
- 如需使用外置 XO 32k，需对外置 XO 32k 时钟源进行时间精度测量，去掉 'SUPPORT_RC_CALIBRATION' 编译宏并添加 'SUPPORT_32K_MEAS' 编译选项开启，串口会周期性打印当前 32k 时钟源频率值，测量后请及时删除 'SUPPORT_32K_MEAS' 编译选项，避免对系统调度造成影响。

具体设置编译选项方法：

步骤 1 进入 “software\code\sdk\build\config\target_config\brandy” 目录，打开 “config.py”。

步骤 2 如需切换 RC 32k 时钟源，按照图 3-4 所示添加对应编译选项进行编译即可。

图3-4 添加编译选项切换 RC 32k 时钟源



```
99 #target3: liteos + uikit + nandflash + mipi + psram + audio 典型合封V100版本
100 'brandy-target3': {
101     'board': 'evb',
102     'base_target_name': 'target_standard_brandy_application_template',
103     'defines': ['PRE_ASIC', 'BRANDY_PRODUCT_EVb4', 'VERSION_STANDARD', '_LITEOS_', 'CONFIG_OTA_UPDATE_SUPPORT',
104               'ALL_SOURCE', 'SUPPORT_CXX', 'I2C_SLAVE_REG_ADDR_4BYTE', 'CONFIG_PSRAM_SUPPORT',
105               'TARGET_CHIP_BRANDY=1', 'BRANDY_CHIP_V100=1', 'CFG_DRIVERS_NANDFLASH', 'CONFIG_ZDIAG_NV_SUPPORT',
106               'CONFIG_ZDIAG_AUDIO_PROC_SUPPORT', 'CONFIG_ZDIAG_AUDIO_DUMP_SUPPORT', 'CONFIG_ZDIAG_AUDIO_PROBE_SUPPORT', 'CONFIG_SEA_PHS_SUPPORT',
107               'SUPPORT_BLE', 'SUPPORT_BREDR', 'ENABLE_UKIT', 'SUPPORT_GPU_JPEG', 'SUPPORT_GPU_GMMU', 'SUPPORT_GPU_OPENVG',
108               'SW_UART_DEBUG', 'SAVE_EXC_INFO', 'HASH_MEM_COPY', 'MIPU_ULPS_SUPPORT', 'CONFIG_LOW_POWER_TEST', 'CONFIG_ZDIAG_GPU_PROC_SUPPORT',
109               'CONFIG_CLD0IPD_SUPPORT', 'ENABLE_ECC', 'SUPPORT_ALIPAY_SEC', 'SUPPORT_RC_CALIBRATION'],
110     'ram_component': ['algorithm', 'dfx_port_brandy', 'dfx_update', 'dfx_nv', 'osal', 'arch_port', 'testsuite',
111                      'liteos_port', 'lcd', 'qspi_display', 'dfx_file_operation',
112                      'psram', 'hal_mipi', 'mipi_tx', 'non_os', 'touch',
113                      'ulp_aon', 'hal_l2ram',
114                      'x_dpai', 'x_vfs', 'x_disk', 'x_fat', 'drv_mmc', 'fs_yaffs2', 'x_vfs_private',
115                      'pm_service', 'brandy', 'cmn_header', 'at_cmd', 'graphic_at_service', 'app_at_service',
116                      'audio_at_service', 'media_at_service', 'bt_manager_at_service',
117                      'audio_proc', 'audio_dump', 'audio_probe', 'gpu_proc', 'bts_header'],
118     'ram_component_set': ['bgh', 'media', 'gpu', 'graphic_bkit', 'graphic_test', 'bgh_audio', 'dfx_sec', 'alipay_set'],
119     'dsp_version': 'max',
120     'image_analysis': True,
121     'packet': True,
122     'fs_image': False
123 }
```

步骤 3 如需切换回 XO 32k 晶体时钟源，删除上述步骤 2 添加的编译选项，重新编译即可。

----结束

3.1.2.9 Nor Flash 部署

SDK 版本提供了 8M 和 16M 两种 Nor Flash 部署方案，版本默认使用 8M Nor Flash。如果客户需要切换到 16M Nor Flash 部署，需要增加 CFG_FLASH_SIZE=16 编译宏。

以 native-js 版本为例，具体操作如下：

步骤 1 brandy-native-js 新增 CFG_FLASH_SIZE=16 宏定义。

步骤 2 brandy-native-js-r 新增 CFG_FLASH_SIZE=16 宏定义。

步骤 3 brandy-ssb 新增 CFG_FLASH_SIZE=16 宏定义。

----结束

3.1.3 各组件编译描述

编译 target 与对应二进制对应关系如表 3-4 所示。

表3-4 编译 target 与 bin 文件对应关系

编译 target	bin 文件	说明
brandy-targetx	application.bin	freertos/liteos 系统的 A 核版本。
brandy-ssb	ssb.bin	用于启动和加载的版本。

3.1.4 编译结果获取

对应二进制获取路径如表 3-5 所示。

表3-5 二进制获取路径

bin 文件	路径
ssb.bin	sdk\output\brandy\core\brandy-ssb
application.bin	sdk\output\brandy\core\brandy-xxx\

3.1.5 Bin 文件描述

表3-6 bin 文件说明

bin 文件	说明
ssb.bin	boot 启动加载所需二进制文件。
application.bin	A 核系统版本二进制文件，可通过 code 目录 SDK 版本增量开发后生成。
bt.bin	bt 核系统二进制文件，固定版本发布，不支持增量开发。

bin 文件	说明
dsp_main.bin/dsp_overlay.bin	dsp 核系统二进制文件，固定版本发布，不支持增量开发。

3.1.6 target 简介

系统默认适配了多个 target，方便用户进行开发和使用，下面对 target 进行简介：

- target1：主要组件有 liteos + lvgl + nandflash + mipi + no psram + mini_audio，是一个使用 LVGL 开发的手表类软件版本，使用的芯片为不带 psram 单封芯片。
- target3：主要组件有 liteos + uikit + nandflash + mipi + psram + audio，是一个使用 UIKIT 开发的手表类软件版本，使用的芯片为带 psram 的合封芯片。
- target4：主要组件有 liteos + nandflash + psram + mini_audio + no gui，是一个心率带类的软件版本，使用的芯片为带 psram 的合封芯片。
- target5：主要组件有 freertos + lvgl + nand + mipi + psram + audio + media，是一个使用 LVGL 开发的手表类软件版本，使用的芯片为带 psram 的合封芯片。
- brandy-target1-r brandy-target3-r brandy-target4-r brandy-target5-r：这些带“-r”的 target 是上述对应 target 的 recovery 升级版本，用来对系统进行升级。
- brandy-native-js：新增了鸿蒙的 JS 组件，可以快速进行上层应用开发。
- brandy-ssb：是一个 SSB 的组件，作用是程序从 ROM 中跳转到 SSB 中，进行部分驱动的启动和升级使用，如无特殊需求，开发用户使用默认版本。

3.1.7 三方库集成编译

库的引入一般是头文件+库，此处以“libsample_b.a”为例。分为两步：

步骤 1 在依赖三方库的组件的“CMakeLists.txt”下，添加头文件依赖，示例如下：

假设将“sample_b.h”放置在 CMakeLists 根路径下。

```
set(SOURCES
    ${CMAKE_CURRENT_SOURCE_DIR}
```

步骤 2 在依赖三方库的组件的“CMakeLists.txt”下，添加三方库，示例如下：

假设将“libsample_b.a”放置在“CMakeLists.txt”根路径下：

```
find_library(LIB_SAMPLE_B libsample_b.a ${CMAKE_CURRENT_SOURCE_DIR})
target_link_libraries(${TARGET_NAME} PRIVATE -Wl,--whole-archive ${LIB_SAMPLE_B}
PRIVATE -Wl,--whole-archive)
```



```
set(PUBLIC_DEFINES
)

set(PRIVATE_DEFINES
)

build_component()

find_library(LIB_SAMPLE_B libsample_b.a ${CMAKE_CURRENT_SOURCE_DIR})
target_link_libraries(${TARGET_NAME} PRIVATE -Wl,--whole-archive ${LIB_SAMPLE_B} PRIVATE -Wl,--whole-archive)
```

----结束

3.2 系统适配对接

3.2.1 系统启动适配

系统启动适配，主要包括 SSB 和 APPLICATION 的适配，其中 SSB 二次开发适配主要参考《HiDiTingV100 Boot 开发指南》，本小节主要阐述 APPLICATION 的适配。当前 APPLICATION 主函数（application/brandy/brandy_standard/main.c）首先完成各类初始化，然后 Os 开始进行任务调度，具体流程如下：

步骤 1 Kernel init：Os kernel 初始化。

步骤 2 Chip init：初始化芯片内部 IP 模块。

步骤 3 Board init：板极总线、外设初始化。

步骤 4 Service init：服务初始化，包括 DFX 等。

步骤 5 Thread init：任务初始化，初始化各个任务。

步骤 6 osKernelStart：触发任务调度。

----结束

当前系统启动适配，首先需要修改 board_init 函数，增加板极器件初始化；然后修改 service_init 函数，增加对应服务初始化；最后在 thread_init 中，新增相应任务即可正常运行。

3.3 调试环境

3.3.1 JLink 环境调试

详情请参见《HiDiTingV100 IDE 工具使用指南》中“工程调试”章节。

3.4 烧录

须知

由于串口时钟与波特率为非整数倍关系时，串口传输存在误差率，一旦发生传输误差将会导致烧录失败。

因此建议烧录时请按照实际串口支持能力，将波特率设置为 4M、2M、1M、500k、250k 等与串口时钟（32MHz）整数倍关系的串口波特率，防止传输误差产生。

系统版本准备就绪后，SDK 提供两种烧录方式：

- 烧录工具进行烧录。

SDK 包 tools 目录下提供 BurnTool 工具，专用于版本烧录，同时支持产测阶段一拖多进行烧写。同时支持基于指令运行的烧写方式。详情请参见《HiDiTingV100 BurnTool 工具 使用指南》。

- IDE 工具集成烧录。

SDK 包 tools 目录下提供 DevEco IDE 工具包，安装后自动支持烧写功能。支持一键式编译后进行烧写。详情请参见《HiDiTingV100 IDE 工具 使用指南》。

两种方式均可独立完成烧录功能，均支持图形化界面对待烧录的二进制文件进行勾选。烧写均基于串口方式进行。

3.4.1 烧录环境说明

SOC 以及 GNSS 整体加载烧录请参见《HiDiTingV100 BurnTool 工具 使用指南》、《HiDiTingV100 IDE 工具 使用指南》。

Hi2131E 烧录详情请见 Hi2131E 目录下《HiDiTingV100 VoLTE 开发指南》2.1 章节中。

3.4.2 烧录步骤说明

详情请参见《HiDiTingV100 BurnTool 工具 使用指南》、《HiDiTingV100 IDE 工具 使用指南》。

3.5 代码部署

基于 SDK 开发时，客户代码加到 SDK 框架进行编译，除了正确书写组件式 CMake 之外，还需规划客户代码存放位置。代码部署通过 ID 文件进行规划，以“A 核 /drivers/boards/brandy_evb/linker/standard/acore.prelds”为例进行说明。

客户代码的运行地址和存储地址建议放在 Flash 中，正则表达式的写法有两种：

- 基于.a 维度，参考图 3-5 中的 1。
- 基于文件路径维度，参考图 3-5 中的 2。

图3-5 代码示例

```
439  .text :
440  {
441      FILL(0xFF)
442      . = ALIGN(8);
443      /* TODO: add close component code here begin */
444      /* new liteos libs config */
445      SORT(*libc.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
446      SORT(*libcmsis.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
447      SORT(*libdrivabase.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
448      SORT(*libinit.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
449      SORT(*libinterrupt.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
450      SORT(*liblinux.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
451      SORT(*libposix.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
452      SORT(*libm.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
453      SORT(*libtargets.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
454
455      1 SORT(*libseggen.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
456      SORT(*libbt_host_sdk.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
457      SORT(*libbt_host_brandy_service.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
458      SORT(*libbt_host_brandy_sample.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
459      SORT(*libbt_test_audio_manager_wrapper.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
460      SORT(*libhmf_demuxer.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
461      SORT(*libplugin_demuxer_hmf.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
462      SORT(*libplugin_demuxer_mjpeg.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
463      SORT(*libplugin_protocol_https.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
464      SORT(*libgfx_rm.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
465      SORT(*libclang_rt.builtins-riscv32.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
466
467      /* TODO: add close component code here end */
468      #ifdef TEST_SUITE
469      2 SORT(*test/common/testsuite.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
470      SORT(*test/platform/testsuite.a:*.*.*) (.text .text* .rodata .rodata* .srodata .srodata*)
471      #endif
472      *(.ramtext)
473      *(.text)
474      *(.text*)
475      *(.flashtext)
476      *(.flashtext*)
477      *(.rodata)
478      *(.rodata*)
479      . = ALIGN(8);
480      > FLASH_PROGRAM
```


4 开发调试工具应用

本次主要提供四款工具，分别用于系统调试维测、版本烧录、系统工程开发、UIKit 开发。

- DebugKits：提供日志打印解析、定制命令输入、内存导出解析等功能。
- BurnTool：提供版本烧录功能。
- DevEco：开发工具，集成一键式编译、烧录、单步调试以及内存查询，变量监控等功能。
- HiSpark Studio 工具：优化版本 IDE，集成一键式编译、烧录、单步调试以及内存查询、变量监控等，以及提供基于 UIKit 组件的 windows 环境下模拟器功能。

4.1 工具概述

4.2 基础开发工具应用介绍

4.3 基础维测工具应用介绍

4.4 业务应用工具介绍

4.1 工具概述

DevEco 南向 IDE

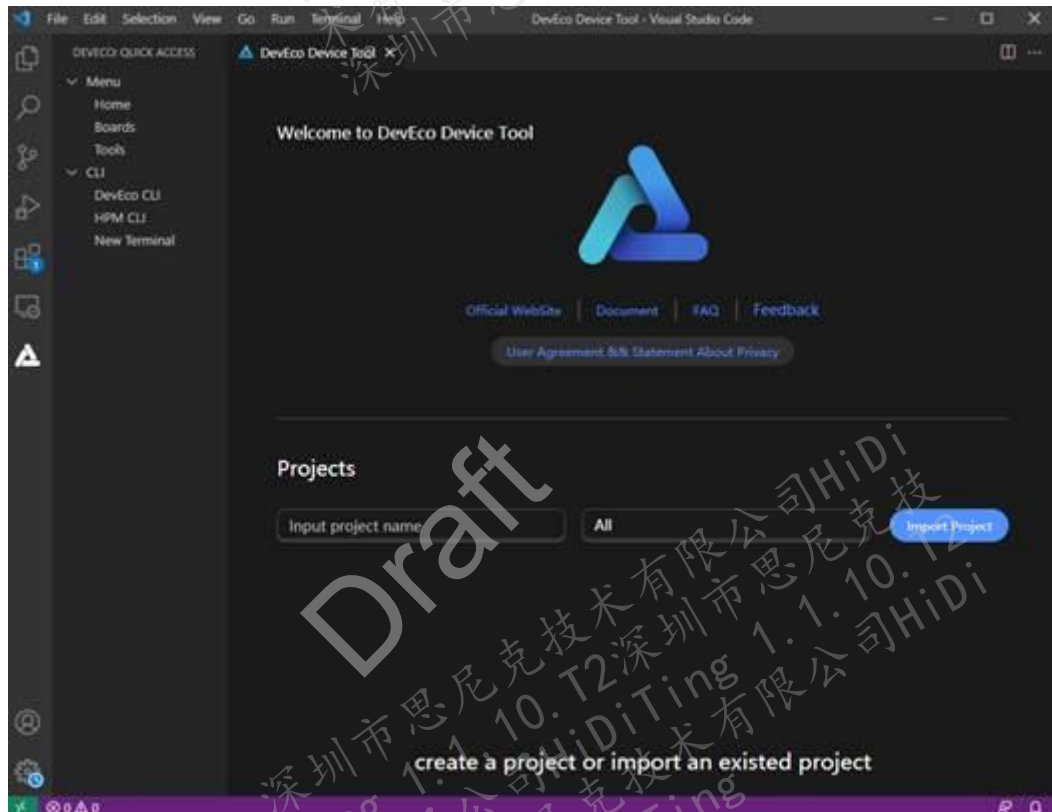
当前支持功能如下：

- 支持工程管理。
- 支持常用编辑功能。
- 集成编译烧写调试。

- 支持镜像分析、栈估算。
- 支持常用调试功能。
- 支持一键安装编译环境。

效果图如图 4-1 所示。

图4-1 DevEco Device Tool 工具界面



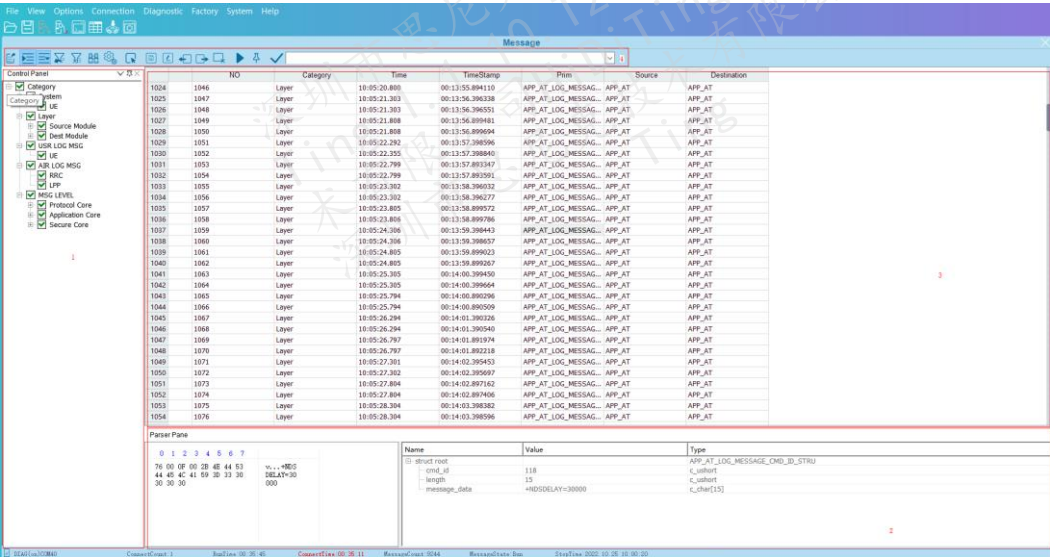
DebugKits

当前支持功能如下：

- 支持日志查看。
- 支持自定义参数配置。
- 支持自定义维测命令。
- 支持多种数据导入导出。
- 支持寄存器读写。
- 支持死机分析。

效果图如图 4-2 所示。

图4-2 消息插件界面示例



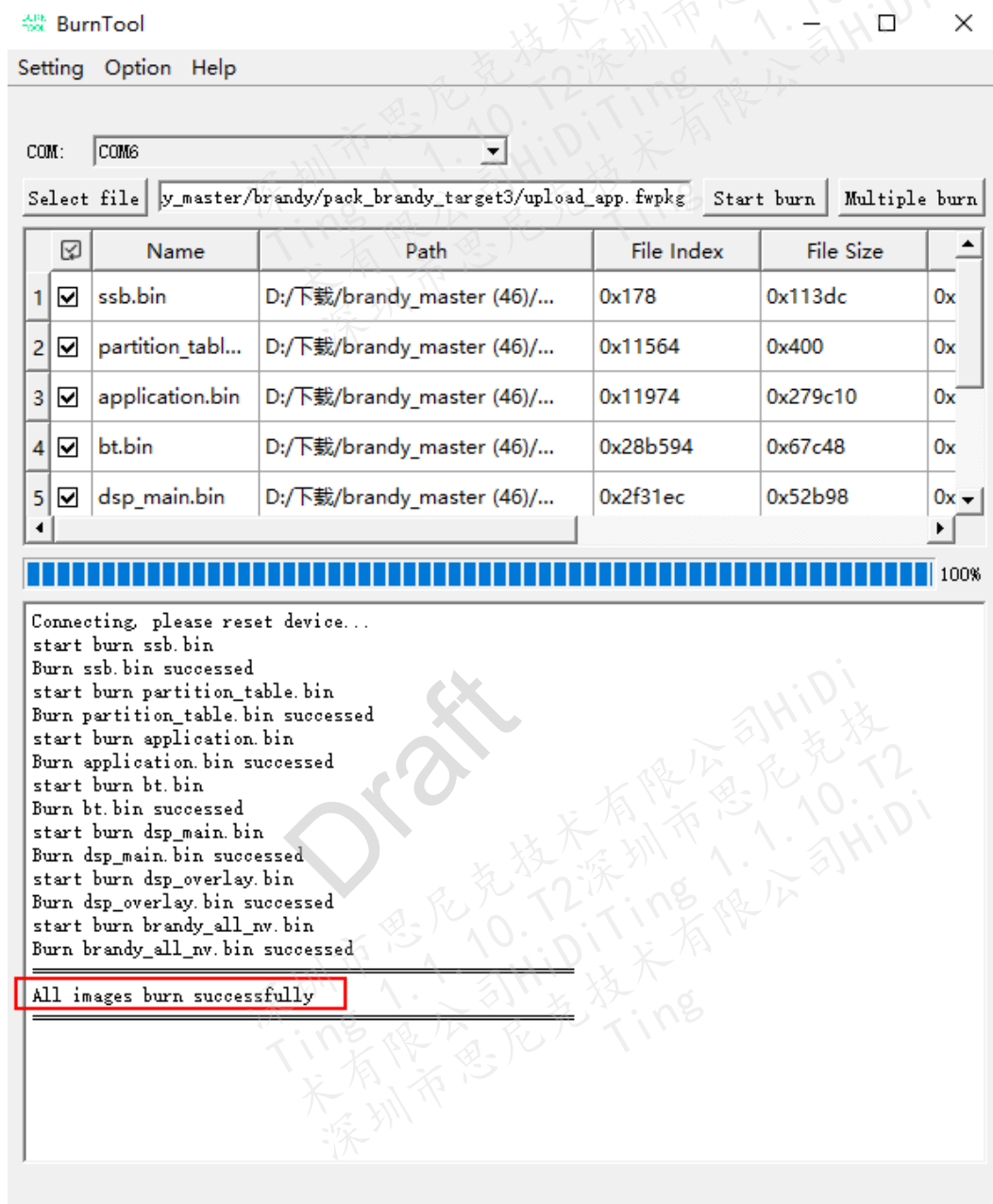
BurnTool

当前支持功能如下：

- 支持多种烧写协议（基于脚本集成）。
- 支持 efuse 读写。
- 支持 flash 导出。
- 支持 cmd 命令行烧写或者 BurnTool GUI 烧写。
- 支持一拖多。

效果图如图 4-3 所示。

图4-3 烧写完成示意图



4.2 基础开发工具应用介绍

HiDiTing 系统解决方案 SDK 提供 DevEco IDE 工具用于 SDK 增量功能开发，提供如下功能：

工具链	当前支持功能
-----	--------

工具链	当前支持功能
DevEco 南向 IDE	• 支持工程管理 • 支持常用编辑功能 • 集成编译烧写调试 • 支持镜像分析、栈估算 • 支持常用调试功能 • 支持一键安装编译环境

详情请参考《HiDiTingV100 IDE 工具使用指南》。

4.3 基础维测工具应用介绍

HiDiTing 系统解决方案 SDK 提供 DebugKits 工具用于基础系统维测，提供如下功能：

- 支持日志解析/查看。
- 支持自定义参数配置（需适配板端 Diag 模块）。
- 支持自定义维测命令（需适配板端 Diag 模块）。
- 支持多种系统内存数据导入导出。
- 支持板端寄存器读写。
- 支持死机分析。

详情请参考《HiDiTingV100 DebugKits 工具 使用指南》

4.4 业务应用工具介绍

业务应用当前共提供四种对应工具。

- 音频工具：提供音频源格式转换等功能。
音频工具主要应用于多媒体业务，针对多媒体业务需要的音频资源进行对应格式转换等处理。当前此工具不单独提供，整体工具集成于 DebugKits 工具中交付使用。详情请参考《HiDiTingV100 DebugKits 工具 使用指南》。
- 图形压缩工具：提供图片资源压缩打包功能。

图形压缩工具主要应用于 UIKIT 图形引擎，针对图形引擎所需要的图形资源进行压缩处理。当前此工具不单独提供，整体工具集成于 DebugKits 工具中交付。详情请参考《HiDiTingV100 DebugKits 工具 使用指南》。

- UIKIT 模拟器：提供基于 UIKIT 的图形模拟效果预览及 SDK 编译烧录调试功能。详情请参考《HiDiTingV100 HiSpark Studio 工具 使用指南》。
- 可视化布局工具：提供基于控件拖曳生成对应 UIKIT 可识别图形代码的功能详情请参考《HiDiTingV100 HiSpark Studio 工具 使用指南》。

5 RTT 调试使用指导

RTT (Real-Time Transfer) 实时跟踪技术是 J-Link 的一个功能，它为嵌入式应用程序提供了一种快速、可靠的调试和日志记录方式。RTT 允许通过 J-Link 直接将应用程序的信息传输到主机上进行显示或日志记录，而无需依赖 UART 或其他串口。

- 5.1 RTT 源码获取
- 5.2 RTT 模块添加
- 5.3 RTT Viewer 设置

5.1 RTT 源码获取

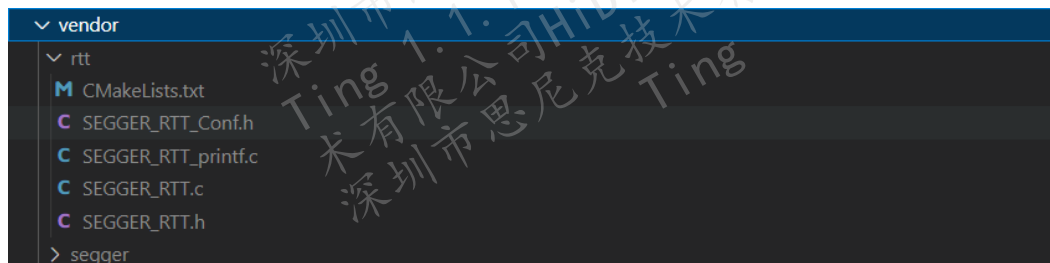
RTT 源码在 SDK 中不交付，需要自行获取，如果用户已安装 SEGGER 的 J-Link，则在 “SEGGER\JLink\Samples\RTT\ SEGGER_RTT_V698c.zip” 获取源码，需要如[图5-1](#)所示文件即可。

图5-1 文件列表

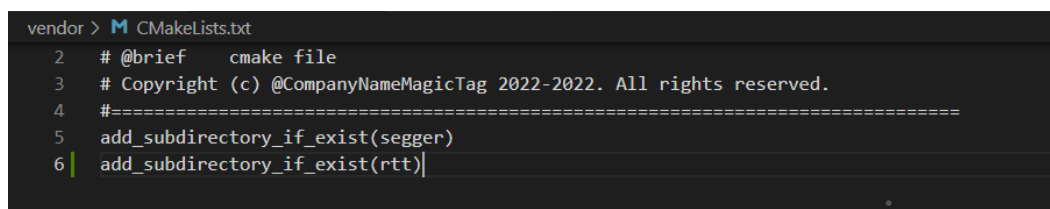
☐ 名称	修改日期	类型	大小
☒ SEGGER_RTT.c	2021/3/16 19:25	C 源文件	73 KB
☒ SEGGER_RTT.h	2021/3/16 19:25	C Header 源文件	21 KB
☐ SEGGER_RTT_ASM_ARMv7M.S	2021/3/2 22:26	S 文件	12 KB
☒ SEGGER_RTT_Conf.h	2021/3/16 19:25	C Header 源文件	26 KB
☒ SEGGER_RTT_printf.c	2021/3/16 19:25	C 源文件	16 KB

5.2 RTT 模块添加

步骤 1 RTT 模块添加至 “software\code\sdk\vendor”，新增 rtt 目录，拷贝 RTT 源码，并且新增编译文件 “CMakeLists.txt”。



步骤 2 在 vendor 目录下的 “CMakeLists.txt” 文件增加编译引用。



步骤 3 编译配置文件中添加 RTT 模块。

在配置文件 “software\code\sdk\build\config\target_config\brandy\config.py” 对应 target 中的 ram_component 中增加 “rtt” 实现 RTT 组件添加，并且修改 “SW_UART_DEBUG” 宏为 “SW_RTT_DEBUG” 完成 RTT 替换 UART 调试。

图5-2 APP target 修改



图5-3 SSB target 修改

```

'brandy-ssb': {
  'base_target_name': 'target_ssb_template_brandy',
  'board': 'evb',
  'defines': [":-:TARGET_CHIP_BRANDY=1", ":-:BRANDY_CHIP_V100=1", "PRE_ASIC", "BRANDY_PRODUCT_EVB",
    "SSB_USE_PLL", "VERSION_STANDARD", "BUILD_APPLICATION_SSB", "LIBBOOTLOADER", "LIBCODELOADER",
    "LIBCPU_UTILS", "LIBAPP_VERSION", "LIBBRANDY", "LIBCONNECTIVITY", "LIBERROR_CODE", "LIBSEC_RANDOM",
    "LIBCODELOADER_SSB", "LIBJLINK_LOAD", "LIBLIB_UTILS", "LIBBUILD_VERSION",
    "NO_TIMEOUT", "CONFIG_UART_BAUD_RATE_DIV_8", "SW_RTT_DEBUG"],
  'ram_component': ['arch_port', 'hal_l2ram', 'sec_boot', 'libboundscheck',
    'dfx_exception', 'error_code', 'psram', 'osal', 'cmn_header', 'boot_nandflash', 'rtt'],
  'ram_component_set': ['pinctrl'],
  'target': 'ssb',
}

```

步骤 4 在步骤 1 新增的 RTT 目录新增编译文件 “CMakeLists.txt”，可参考如下代码示例：

```

#=====
=====
# @brief    cmake file
# Copyright (c) CompanyNameMagicTag 2022-2023. All rights reserved.
#=====
=====
set(COMPONENT_NAME "rtt")

set(SOURCES
    ${CMAKE_CURRENT_SOURCE_DIR}/SEGGER_RTT.c
    ${CMAKE_CURRENT_SOURCE_DIR}/SEGGER_RTT_printf.c
)

set(PUBLIC_HEADER
    ${CMAKE_CURRENT_SOURCE_DIR}/
)

set(PRIVATE_HEADER
)

set(PRIVATE_DEFINES
)

set(PUBLIC_DEFINES
)

# use this when you want to add ccflags like -include xxx
set(COMPONENT_PUBLIC_CCFLAGS
)

set(COMPONENT_CCFLAGS
)

```

```
)  
  
set(WHOLE_LINK  
    true  
)  
  
set(MAIN_COMPONENT  
    false  
)  
  
build_component()
```

----结束

📖 说明

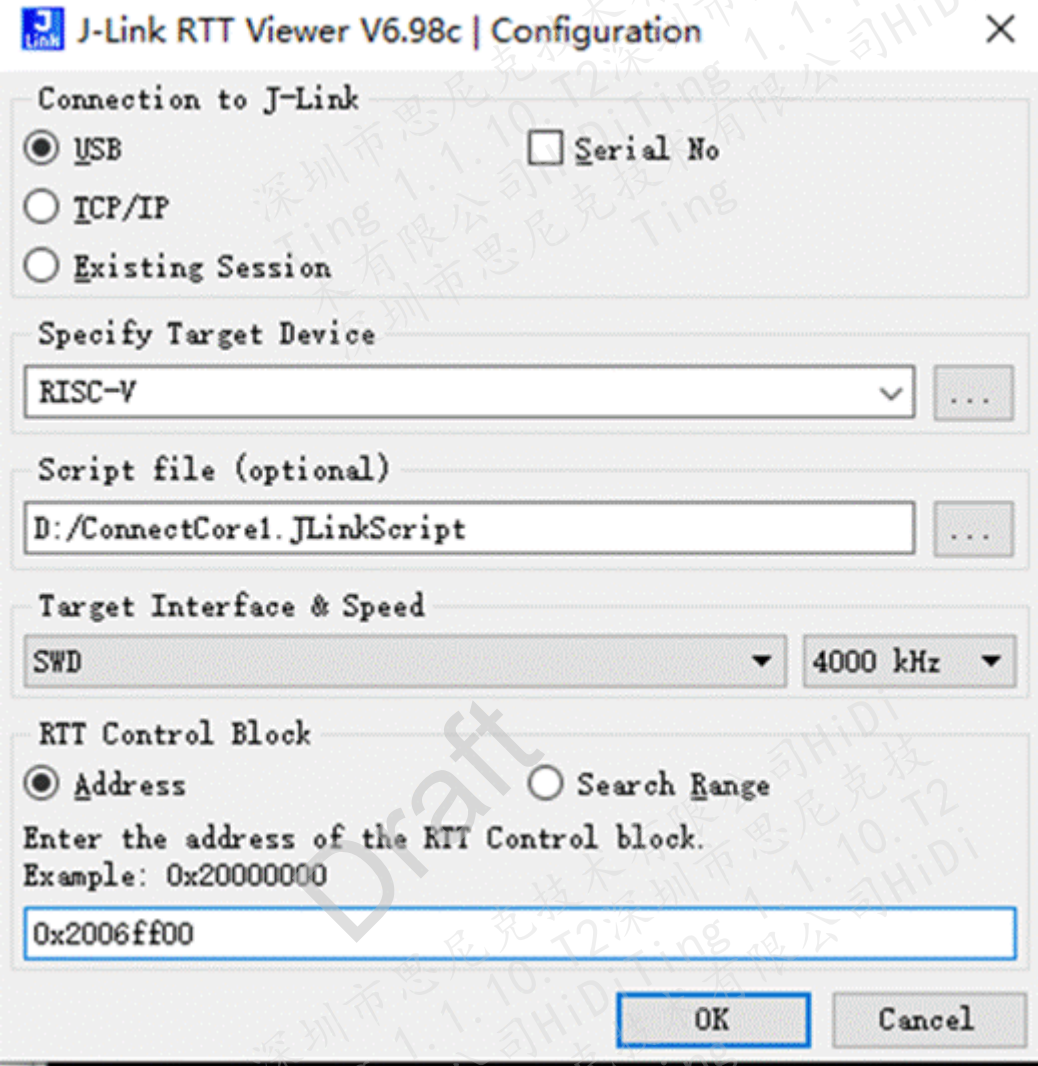
RTT 不支持低功耗，在使用 RTT 作为输入输出时，需要关闭低功耗，具体方法请参见《HiDiTingV100 低功耗软件 开发指南》中的“特性配置”章节。

5.3 RTT Viewer 设置

步骤 1 RTT 链接配置：

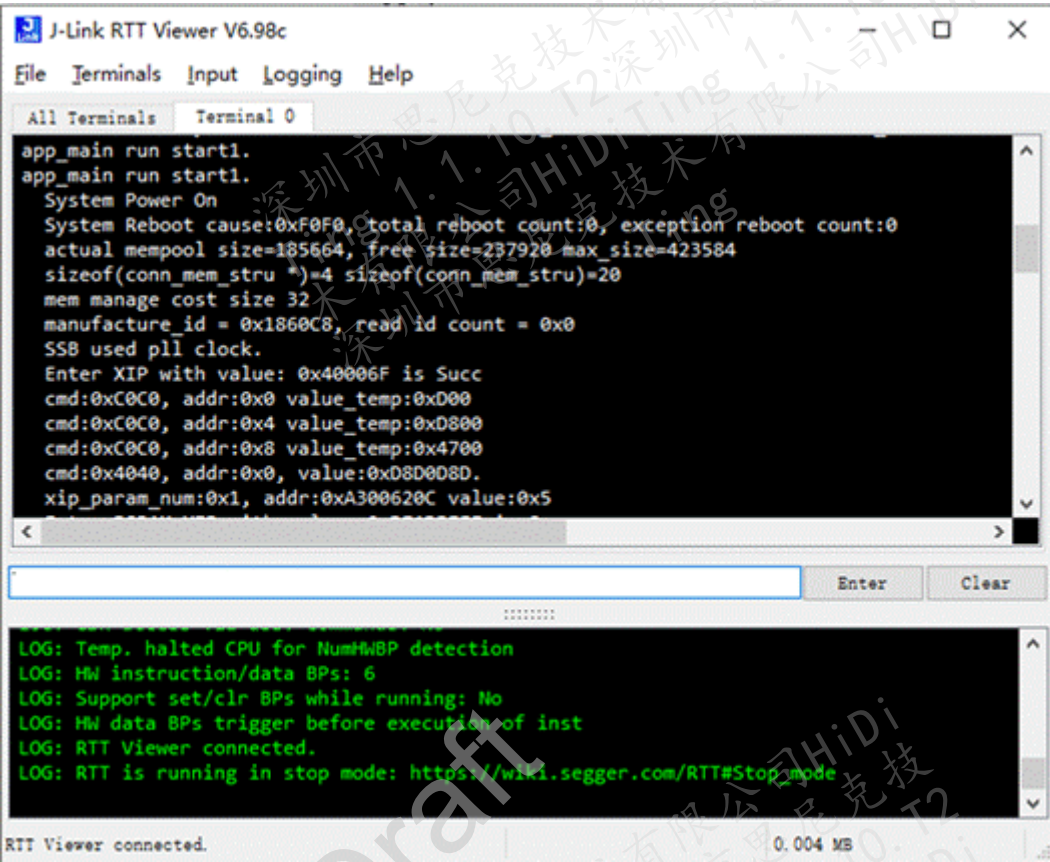
- Connection to J-Link: USB
- Specify Target Device: RISC-V
- Script file:
customer\tools\DevEco\DevTools_CFBB_V1.0.3\jlink_script\ConnectCore1.JLink
Script
- RTT Control block: address 0x2006FF00

图5-4 RTT 链接配置示意图



步骤 2 版本运行后，配置 RTT Viewer，日志信息会在 RTT 界面打印。

图5-5 RTT 界面打印信息



----结束

说明

RTT Control block 地址固定设置为 0x2006FF00。